QUESTION 1

Server code

```
#include <stdio.h>
#include <string.h>
#include <winsock2.h>
#define PORT 8080
int main()
{
   WSADATA wsa;
   SOCKET serverSocket, clientSocket;
   struct sockaddr_in serverAddr, clientAddr;
   int addrSize;
   char buffer[1024];
   WSAStartup(MAKEWORD(2, 2), &wsa);
   serverSocket = socket(AF_INET, SOCK_STREAM, 0);
   serverAddr.sin_family = AF_INET;
   serverAddr.sin_addr.s_addr = INADDR_ANY;
   serverAddr.sin_port = htons(PORT);
   bind(serverSocket, (struct sockaddr *)&serverAddr,
      sizeof(serverAddr));
   listen(serverSocket, 5);
   printf("Server is waiting for connection...\n");
   addrSize = sizeof(clientAddr);
   clientSocket = accept(serverSocket,
                  (struct sockaddr *)&clientAddr,
                  &addrSize);
   recv(clientSocket, buffer, sizeof(buffer), 0);
```

```
    printf("Client says: %s\n", buffer);
    strcpy(buffer, "Hello Client");
    send(clientSocket, buffer, strlen(buffer) + 1, 0);
    closesocket(clientSocket);
    closesocket(serverSocket);
    WSACleanup();
return 0;
}
```

Client

```
#include <stdio.h>
#include <string.h>
#include <winsock2.h>

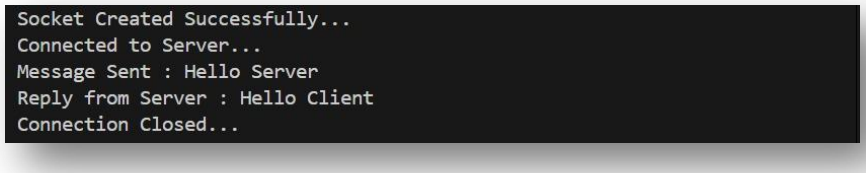
#define PORT 8080

int main()
{
    WSADATA wsa;
    SOCKET clientSocket;
    struct sockaddr_in serverAddr;
    char buffer[1024];
    WSAStartup(MAKEWORD(2, 2), &wsa);
    clientSocket = socket(AF_INET, SOCK_STREAM, 0);
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
    serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");
    connect(clientSocket,
        (struct sockaddr *)&serverAddr,
        sizeof(serverAddr));
    strcpy(buffer, "Hello Server");
    send(clientSocket, buffer, strlen(buffer) + 1, 0);
    recv(clientSocket, buffer, sizeof(buffer), 0);

    printf("Server says: %s\n", buffer);
    closesocket(clientSocket);
```

```
    WSACleanup();
    return 0;
}
```

OUTPUT





QUESTION 2

SERVER

```
#include <stdio.h>
#include <winsock2.h>

#define PORT 8080

int main()
{
    WSADATA wsa;
    SOCKET serverSocket, clientSocket;
    struct sockaddr_in serverAddr, clientAddr;
    int addrSize;

    int a, b;
    int sum, difference, product;
    float division;

    // Initialize Winsock
    WSAStartup(MAKEWORD(2, 2), &wsa);

    // Create socket
    serverSocket = socket(AF_INET, SOCK_STREAM, 0);

    // Server address
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_addr.s_addr = INADDR_ANY;
```

```
    serverAddr.sin_port = htons(PORT);

    // Bind
    bind(serverSocket, (struct sockaddr *)&serverAddr,
        sizeof(serverAddr));

    // Listen
    listen(serverSocket, 5);

    printf("Server is waiting for connection...\n");

    // Accept client
    addrSize = sizeof(clientAddr);

    clientSocket = accept(serverSocket,
                    (struct sockaddr *)&clientAddr,
                    &addrSize);

    // Receive two integers
    recv(clientSocket, (char *)&a, sizeof(a), 0);
    recv(clientSocket, (char *)&b, sizeof(b), 0);

    // Perform calculations
    sum = a + b;
    difference = a - b;
    product = a * b;

    if (b != 0)
       division = (float)a / b;
    else
       division = 0;

    // Send results
    send(clientSocket, (char *)&sum, sizeof(sum), 0);
    send(clientSocket, (char *)&difference, sizeof(difference), 0);
    send(clientSocket, (char *)&product, sizeof(product), 0);
    send(clientSocket, (char *)&division, sizeof(division), 0);

    closesocket(clientSocket);
    closesocket(serverSocket);

    WSACleanup();

    return 0;
}
```

CLIENT

```
#include <stdio.h>
#include <winsock2.h>

#define PORT 8080

int main()
{
   WSADATA wsa;
   SOCKET clientSocket;
   struct sockaddr_in serverAddr;
   int a, b;
   int sum, difference, product;
   float division;
   WSAStartup(MAKEWORD(2, 2), &wsa);
   clientSocket = socket(AF_INET, SOCK_STREAM, 0);
   // Server address
   serverAddr.sin_family = AF_INET;
   serverAddr.sin_port = htons(PORT);
   serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");
   // Connect to server
   connect(clientSocket,
         (struct sockaddr *)&serverAddr,
         sizeof(serverAddr));
   // Input
   printf("Enter two integers: ");
   scanf("%d %d", &a, &b);

   // Send integers
   send(clientSocket, (char *)&a, sizeof(a), 0);
   send(clientSocket, (char *)&b, sizeof(b), 0);

   // Receive results
   recv(clientSocket, (char *)&sum, sizeof(sum), 0);
   recv(clientSocket, (char *)&difference, sizeof(difference), 0);
   recv(clientSocket, (char *)&product, sizeof(product), 0);
   recv(clientSocket, (char *)&division, sizeof(division), 0);

   // Display results
   printf("\nAddition      = %d\n", sum);
   printf("Subtraction    = %d\n", difference);
   printf("Multiplication = %d\n", product);

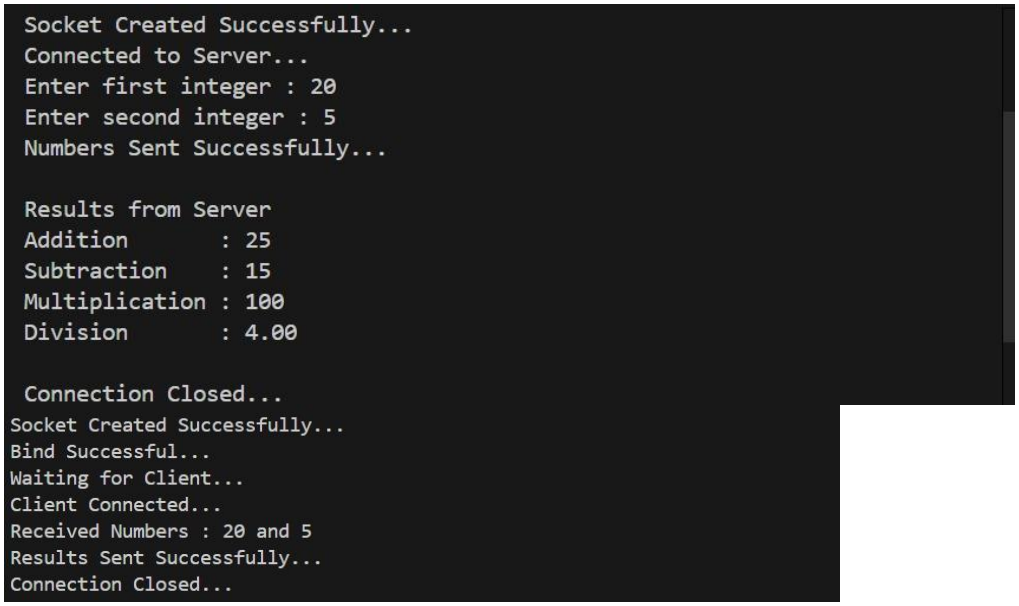
   if (b != 0)
      printf("Division      = %.2f\n", division);
   else
      printf("Division      = Cannot divide by zero\n");
```

```
    closesocket(clientSocket);

    WSACleanup();

    return 0;
}
```

OUTPUT



QUESTION 3

SERVER

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>

#include <winsock2.h>

#define PORT 8080

int main()

{

    WSADATA wsa;

    SOCKET serverSocket, clientSocket;

    struct sockaddr_in serverAddr, clientAddr;

    int addrSize;
```

```
char str[1024];
char uppercase[1024];
char reverse[1024];
int length;
int i;
WSAStartup(MAKEWORD(2, 2), &wsa);
serverSocket = socket(AF_INET, SOCK_STREAM, 0);
serverAddr.sin_family = AF_INET;
serverAddr.sin_addr.s_addr = INADDR_ANY;
serverAddr.sin_port = htons(PORT);
bind(serverSocket, (struct sockaddr *)&serverAddr,
    sizeof(serverAddr));
listen(serverSocket, 5);
printf("Server is waiting for connection...\n");
addrSize = sizeof(clientAddr);
clientSocket = accept(serverSocket,
                (struct sockaddr *)&clientAddr,
                &addrSize);
recv(clientSocket, str, sizeof(str), 0);
length = strlen(str);
for (i = 0; i < length; i++)
{
   uppercase[i] = toupper((unsigned char)str[i]);
}
uppercase[length] = '\0';
for (i = 0; i < length; i++)
{
   reverse[i] = str[length - 1 - i];
```

```
    }
    reverse[length] = '\0';
    send(clientSocket, (char *)&length, sizeof(length), 0);
    send(clientSocket, uppercase, sizeof(uppercase), 0);
    send(clientSocket, reverse, sizeof(reverse), 0);
    closesocket(clientSocket);
    closesocket(serverSocket);
    WSACleanup();
    return 0;
}
```

CLIENT

```
#include <stdio.h>
#include <string.h>
#include <winsock2.h>
#define PORT 8080
int main()
{
    WSADATA wsa;
    SOCKET clientSocket;
    struct sockaddr_in serverAddr;
    char str[1024];
    char uppercase[1024];
    char reverse[1024];
    int length;

    // Initialize Winsock
    WSAStartup(MAKEWORD(2, 2), &wsa);
    // Create socket
```

```
clientSocket = socket(AF_INET, SOCK_STREAM, 0);
// Server address
serverAddr.sin_family = AF_INET;
serverAddr.sin_port = htons(PORT);
serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");
// Connect to server
connect(clientSocket,
    (struct sockaddr *)&serverAddr,
    sizeof(serverAddr));
// Input string
printf("Enter a string: ");
fgets(str, sizeof(str), stdin);
// Remove newline
str[strcspn(str, "\n")] = '\0';
// Send string
send(clientSocket, str, sizeof(str), 0);
// Receive results
recv(clientSocket, (char *)&length, sizeof(length), 0);
recv(clientSocket, uppercase, sizeof(uppercase), 0);
recv(clientSocket, reverse, sizeof(reverse), 0);
// Display results
printf("\nLength    = %d\n", length);
printf("Uppercase = %s\n", uppercase);
printf("Reverse   = %s\n", reverse);

closesocket(clientSocket);
WSACleanup();
return 0;
```

}

OUTPUT

```
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Received String : Hello World
Processed Results Sent Successfully...
Connection Closed...
```

```
Socket Created Successfully...
Connected to Server...
Enter a string : Hello World
String Sent Successfully...

Results from Server
String Length : 11
Uppercase     : HELLO WORLD
Reversed      : dlroW olleH

Connection Closed...
```

QUESTION 4

SERVER

```
#include <stdio.h>
#include <stdlib.h>
#include <winsock2.h>
#define PORT 8080
#define MAX 100
int main()
{
   WSADATA wsa;
   SOCKET serverSocket, clientSocket;
   struct sockaddr_in serverAddr;
   int n;
   int arr[MAX];
   int sum = 0;
   int maximum, minimum;
```

```
float average;
// Initialize Winsock
if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
{
   printf("WSAStartup Failed\n");
   return 1;
}
// Create Socket
serverSocket = socket(AF_INET, SOCK_STREAM, 0);
if (serverSocket == INVALID_SOCKET)
{
   printf("Socket Creation Failed\n");
   WSACleanup();
   return 1;
}
printf("Socket Created Successfully...\n");
// Server Address
serverAddr.sin_family = AF_INET;
serverAddr.sin_addr.s_addr = INADDR_ANY;
serverAddr.sin_port = htons(PORT);
// Bind
if (bind(serverSocket,
      (struct sockaddr *)&serverAddr,
      sizeof(serverAddr)) == SOCKET_ERROR)
{
   printf("Bind Failed\n");
   closesocket(serverSocket);
   WSACleanup();
```

```
    return 1;
}
printf("Bind Successful...\n");
// Listen
listen(serverSocket, 5);
printf("Waiting for Client...\n");
// Accept
clientSocket = accept(serverSocket, NULL, NULL);
if (clientSocket == INVALID_SOCKET)
{
    printf("Accept Failed\n");
    closesocket(serverSocket);
    WSACleanup();
    return 1;
}
printf("Client Connected...\n");
// Receive N
recv(clientSocket, (char *)&n, sizeof(n), 0);
// Receive array
recv(clientSocket, (char *)arr, sizeof(int) * n, 0);
printf("Received Array : ");
for (int i = 0; i < n; i++)
{
    printf("%d ", arr[i]);
}
printf("\n");
// Calculate sum
for (int i = 0; i < n; i++)
```

```
    {
        sum += arr[i];
    }
    // Find maximum and minimum
    maximum = arr[0];
    minimum = arr[0];
    for (int i = 1; i < n; i++)
    {
        if (arr[i] > maximum)
            maximum = arr[i];
        if (arr[i] < minimum)
            minimum = arr[i];
    }
    average = (float)sum / n;
    // Send results
    send(clientSocket, (char *)&sum, sizeof(sum), 0);
    send(clientSocket, (char *)&average, sizeof(average), 0);
    send(clientSocket, (char *)&maximum, sizeof(maximum), 0);
    send(clientSocket, (char *)&minimum, sizeof(minimum), 0);
    printf("Results Sent Successfully...\n");
    // Close
    closesocket(clientSocket);
    closesocket(serverSocket);
    printf("Connection Closed...\n");
    WSACleanup();
    return 0;
}
```

Client

```
#include <stdio.h>
#include <stdlib.h>
#include <winsock2.h>
#define PORT 8080
#define MAX 100
int main()
{
   WSADATA wsa;
   SOCKET sockfd;
   struct sockaddr_in serverAddr;
   int n;
   int arr[MAX];
   int sum;
   float average;
   int maximum;
   int minimum;
   // Initialize Winsock
   if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
   {
      printf("WSAStartup Failed\n");
      return 1;
   }
   // Create Socket
   sockfd = socket(AF_INET, SOCK_STREAM, 0);
   if (sockfd == INVALID_SOCKET)
   {
      printf("Socket Creation Failed\n");
```

```
    WSACleanup();
    return 1;
}
printf("Socket Created Successfully...\n");
// Server Address
serverAddr.sin_family = AF_INET;
serverAddr.sin_port = htons(PORT);
serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");
// Connect
if (connect(sockfd,
        (struct sockaddr *)&serverAddr,
        sizeof(serverAddr)) == SOCKET_ERROR)
{
    printf("Connection Failed\n");
    closesocket(sockfd);
    WSACleanup();
    return 1;
}
printf("Connected to Server...\n");
// Input N
printf("Enter number of elements : ");
scanf("%d", &n);
// Input array
printf("Enter %d integers:\n", n);
for (int i = 0; i < n; i++)
{
    scanf("%d", &arr[i]);
}
```

```
    // Send N
    send(sockfd, (char *)&n, sizeof(n), 0);
    // Send array
    send(sockfd, (char *)arr, sizeof(int) * n, 0);
    printf("Array Sent Successfully...\n");
    // Receive results
    recv(sockfd, (char *)&sum, sizeof(sum), 0);
    recv(sockfd, (char *)&average, sizeof(average), 0);
    recv(sockfd, (char *)&maximum, sizeof(maximum), 0);
    recv(sockfd, (char *)&minimum, sizeof(minimum), 0);
    // Display results
    printf("\nResults from Server\n");
    printf("Sum     : %d\n", sum);
    printf("Average : %.2f\n", average);
    printf("Maximum : %d\n", maximum);
    printf("Minimum : %d\n", minimum);
    // Close
    closesocket(sockfd);
    printf("\nConnection Closed...\n");
    WSACleanup();
    return 0;
}
```

OUTPUT

```
Results from Server
Sum     : 155
Average : 25.83
Maximum : 50
Minimum : 5

Connection Closed...
```

```
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Received Array : 5 10 20 30 40 50
Results Sent Successfully...
Connection Closed...
```

Question 5

Server

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>
#define PORT 8080
#define MAX 10
Int main()
{
   WSADATA wsa;
   SOCKET serverSocket, clientSocket;
   Struct sockaddr_in serverAddr;
   Int n, matrix[MAX][MAX];
   Int I, j;
   Int sum = 0, trace = 0;
```

```
Int upper = 1, lower = 1, diagonal = 1;
Char result[1024];
WSAStartup(MAKEWORD(2, 2), &wsa);
serverSocket = socket(AF_INET, SOCK_STREAM, 0);
if (serverSocket == INVALID_SOCKET)
{
   Printf("Socket Creation Failed\n");
   Return 1;
}
Printf("Socket Created Successfully...\n");
serverAddr.sin_family = AF_INET;
serverAddr.sin_addr.s_addr = INADDR_ANY;
serverAddr.sin_port = htons(PORT);
if (bind(serverSocket,
      (struct sockaddr *)&serverAddr,
      Sizeof(serverAddr)) == SOCKET_ERROR)
{
   Printf("Bind Failed\n");
   Closesocket(serverSocket);
   WSACleanup();
   Return 1;
}
Printf("Bind Successful...\n");
Listen(serverSocket, 5);
Printf("Waiting for Client...\n");
clientSocket = accept(serverSocket, NULL, NULL);
if (clientSocket == INVALID_SOCKET)
{
```

```
   Printf(“Accept Failed\n”);
   Return 1;
}
Printf(“Client Connected...\n”);
Recv(clientSocket, (char *)&n, sizeof(n), 0);
Recv(clientSocket, (char *)matrix, sizeof(matrix), 0);
Printf(“\nReceived Matrix:\n”);
For (I = 0; I < n; i++)
{
   For (j = 0; j < n; j++)
   {
      Printf(“%d “, matrix[i][j]);
      Sum += matrix[i][j];
      If (I == j)
         Trace += matrix[i][j];
      If (I != j && matrix[i][j] != 0)
         Diagonal = 0;
      If (I > j && matrix[i][j] != 0)
         Upper = 0;
      If (I < j && matrix[i][j] != 0)
         Lower = 0;
   }
   Printf(“\n”);
}

If (diagonal)
   Strcpy(result, “Matrix Type : Diagonal Matrix\n”);
Else if (upper)
```

```
    Strcpy(result, “Matrix Type : Upper Triangular Matrix\n”);
  Else if (lower)
    Strcpy(result, “Matrix Type : Lower Triangular Matrix\n”);
  Else
    Strcpy(result, “Matrix Type : General Matrix\n”);
  Char temp[200];
  Sprintf(temp, “Sum of Elements : %d\nTrace : %d\n”,
      Sum, trace);
  Strcat(result, temp);
  Printf(“\n%s”, result);
  Send(clientSocket, result, strlen(result) + 1, 0);
  Closesocket(clientSocket);
  Closesocket(serverSocket);
  WSACleanup();
  Return 0;
}
```

Client

```
#include <stdio.h>
#include <stdlib.h>
#include <winsock2.h>
#define PORT 8080
#define MAX 10
Int main()
{
  WSADATA wsa;
  SOCKET clientSocket;
```

```
Struct sockaddr_in serverAddr;
Int n, matrix[MAX][MAX];
Int I, j;
Char result[1024];
WSAStartup(MAKEWORD(2, 2), &wsa);
clientSocket = socket(AF_INET, SOCK_STREAM, 0);
if (clientSocket == INVALID_SOCKET)
{
   Printf(“Socket Creation Failed\n”);
   Return 1;
}
Printf(“Socket Created Successfully...\n”);
serverAddr.sin_family = AF_INET;
serverAddr.sin_port = htons(PORT);
serverAddr.sin_addr.s_addr = inet_addr(“127.0.0.1”);
if (connect(clientSocket,
        (struct sockaddr *)&serverAddr,
        Sizeof(serverAddr)) == SOCKET_ERROR)
{
   Printf(“Connection Failed\n”);
   Closesocket(clientSocket);
   WSACleanup();
   Return 1;
}
Printf(“Connected to Server...\n”);
Printf(“Enter order of square matrix: “);
Scanf(“%d”, &n);
```

```
    Printf("Enter matrix elements:\n");
    For (I = 0; I < n; i++)
    {
      For (j = 0; j < n; j++)
      {
        Scanf("%d", &matrix[i][j]);
      }
    }
    Send(clientSocket, (char *)&n, sizeof(n), 0);
    Send(clientSocket, (char *)matrix, sizeof(matrix), 0);
    Recv(clientSocket, result, sizeof(result), 0);
    Printf("\nResult from Server:\n");
    Printf("%s", result);
    Closesocket(clientSocket);
    WSACleanup();
    Return 0;
}
```

OUTPUT

```
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Received Matrix:
1 0 0
0 0 0
0 0 0
Matrix Type : Diagonal Matrix
Sum : 1
Trace : 1
Results Sent Successfully...
Connection Closed...
```

```
Socket Created Successfully...
Connected to Server...
Enter Matrix Order : 3
Enter Matrix Elements:
1 0 0 0 2 0 0 0 3

Matrix:
1 0 0
0 2 0
0 0 3

Matrix Sent Successfully...

Results from Server
Matrix Type : Diagonal Matrix
Sum         : 1
Trace       : 1

Connection Closed...
```